

个人实验报告（实验三）

桌面物体自动分类整理——仿真执行（mechArm + Gazebo, 阶段一）

班级2024 届中外 2 班姓名马晨曦学号24020036047

小组角色任务控制与系统集成 · 真机执行主责日期2026-09-24

1 实验的基本要求

本实验为机器人集成小组项目 I 《桌面物体自动分类整理》，要求在实验一（视觉检测模型）与实验二（机械臂定点抓取）的基础上，做出一个“看得见—判得准—抓得起—放得对”的全自动整理系统：相机看桌面，识别出网格里摆放的多类物体，机器人依次把每个物体按类别放进对应料盒，直到桌面清空。

实验要求分为**阶段一仿真开发与阶段二真机验证**两段，且 **阶段二的准入条件 = 阶段一仿真验收通过**（顺序不可颠倒）。本人承担的是**阶段一仿真线**——mechArm 270 机械臂 + Gazebo + ROS 2 Humble，负责场景搭建与联调、检测链路接入、取放执行链、仿真演示视频，以及仿真与真机的尺寸对齐。**本报告围绕这条线展开。**

本版范围声明：本报告只写本人承担的**仿真线**。阶段二真机（RoboMaster EP）由组内其他同学承担，本版不写真机的标定数据与运行结论，也不代填任何估计值。两条线唯一共享的是**任务控制核心 sort_core、接口契约 PickPlace 与原因枚举 / 日志 schema**——这部分由本人建立、两线共用（见 §3.1），因此本报告在讲它们时说的是**这套核心本身**，与跑在哪条线上无关。

阶段一仿真的验收结果（本人的主线成果）

仿真线（mechArm + Gazebo, ROS 2）的**记分轮**已于 2026-09-24 跑通，

格号	类别	目标料盒	落料槽位	结果
c1	cup	bin_cup	0	成功
c2	mouse	bin_mouse	0	成功
c3	cup	bin_cup	1	成功
c4	cup	bin_cup	2（第 2 层）	成功
c5	mouse	bin_mouse	1（叠层）	成功
c6	cup	bin_cup	3	失败（放置偏差 52/87/17 mm）

结果文件 `exp3_logs/run_20260924_112831/result.json` 给出 `verdict=PASS`：场上物体 6 个，正确放入对应料盒 5 个（及格线 5），**规划失败 0 次**。

异常轮 (run_20260924_114058) 单独跑：c3 一格故意空置，`verdict=TRIAL`。空网格被跳过的证据不是某一行写了什么，而是 `records.jsonl` 里**没有 c3 这一行**、`result.json`

的 `reasons` 里也没有 `c3` 的键；其余 5 个全部放置成功。这一轮必然是 TRIAL 而非 PASS——判定规则只对**完整**的验收轮下结论，所以异常轮必须与记分轮分开跑。

口径说明（必须分开陈述）。本次仿真跑在没有 GPU 的云容器里，Gazebo 走软件渲染，跑到第 3~4 轮会出现**画面停滞**：帧戳照常推进而画面内容一字不变，依赖图像的 HSV 检测因此失效。为把故障隔离在渲染这一段、继续验证下游链路，记分轮改用**真值投影**生成检测框：物块世界坐标经同一份已标定的相机参数投影为像素，再走完全相同的“像素 → 桌面 → 格子”映射，相机标定与映射链路未作任何改动。因此这一轮**能证明**的是相机标定、投影与网格映射正确，以及规划—取放—落料—回航这条执行链可行；**不能**声称 HSV 检测算法在整轮任务中稳定有效——渲染健康的第 1 轮 HSV 实测 6/6 正确映射，这份证据单独保留。

遗留：叠第 2 层时物块之间发生互穿，LCP 求解器迭代数冲到 122847 并把已放好的物块弹飞；第 2 层落差 $dz=0.036\text{ m}$ 仅比杯高 0.035 m 多 1 mm ，加大落差或让层间 `xy` 错开是下一步要改的地方。`c6` 是记分轮唯一失败项。

仿真任务（docx § 五）

- 在 Gazebo 场景中搭建**取物网格（2×3 共 6 格）**与**2 个分类料盒**，摆放**2 类共 6 个物体**（4 个 `cup` + 2 个 `mouse`）；
- 通过**检测链路**识别物体类别与所在网格——HSV `color_detector` 或真值投影 `truth_detector` (§3.4)；
- 用**单条 Launch** 启动相机、识别、机械臂与任务控制，把不同类别放入对应料盒。

验收标准（docx § 六）

- 6 个物体中**至少 5 个**被正确分类整理；
- 正常任务过程中不得进行人工类别判断和动作触发；
- 正确处理**至少 2 类异常**（空网格、未识别物体应被跳过；抓取失败或目标不可达时应返回安全位置或停止）；
- 提供一个入口启动图像、识别、机械臂和任务控制；
- 全程保存**识别、抓取和任务状态日志**。

仿真平台的确定

docx § 实施原则写的是“真实相机、Jetson 和 `mechArm`”。阶段一**正是这条款的原生形态**：

- **机械臂**: mechArm 270 (6 自由度) , 由 mecharm_grasp 接入 Gazebo, 走 joint_trajectory_controller; 逆解用 c4_2/core/planner.py 的 Newton + 数值 Jacobian——关节限位里 q5 上限 2.0071 rad 是搬运段最常撞到的一条 (§4 问题 2);
- **相机**: Gazebo 的 overhead_camera (俯瞰, 640×480, 10 Hz), 经**相机标定**把像素映射到桌面坐标系 (task.json → camera.calibration);
- **运行环境**: AutoDL 云机容器, **无 GPU**, Mesa 走 llvmpipe 软件渲染。这一点后面直接决定了检测链路的选型 (§3.4)。

注: 本人在这条线上承担**任务控制 (状态机) / 接口契约 / 日志与验收口径**这条主线, 并负责**仿真场景搭建与联调、取放执行链、仿真演示视频**, 以及**仿真与真机的尺寸对齐**。检测模型来自实验一的公开仓库; 阶段二真机 (RoboMaster EP) 的执行侧由组内其他同学承担。

2 个人分工

本人在阶段一仿真线负责以下五件事:

1. **仿真场景与单 Launch**: Gazebo world 里搭出 2×3 取物网格、2 个分类料盒与俯瞰相机, 并把相机、识别、机械臂、任务控制**收进一条** `ros2 launch exp3_sim exp3_sim.launch.py` (docx § 六最后一条要求);
2. **取放执行链 GazeboFollowExecutor**: 实现执行契约的**六段动作** (下探/夹取/抬离/搬运/松放/退回), 并按段号映射为动作级原因; 其中 **同料盒多物错位落料与搬运高度阶梯**是让 6 个物体能全部放下的关键 (§4);
3. **检测链路接入与真值投影检测器**: 在容器无 GPU、Gazebo 软渲染会卡顿的条件下, 把检测框的生成换一种来源而**不动相机标定与映射链路** (§3.4);
4. **任务控制核心 sort_core / 接口契约 PickPlace / 原因枚举与日志 schema**: 这套核心是本人建立的, **两条线共用同一份**, 真机线只是把执行侧换成 EP, 核心一行未改 (见 §3.1);
5. **仿真运行与取证**: 三轮实跑 (记分轮 / 异常轮 / 录屏轮)、演示视频录制, 以及证据目录与验收日志的整理。

组内分工概览: **本人** = 仿真场景搭建与联调 + 取放执行链 + 仿真演示视频 + 仿真与真机的尺寸对齐 + 任务控制与契约; **组内其他同学** = 阶段二真机 (RoboMaster EP) 执行侧、识别链路 (真模型) 与真机相机标定。

3 项目实现的原理

3.1 双线架构：一套任务核心，两个注入缝隙

任务控制核心 `TaskController` 只依赖两个函数缝隙 `scan()`（报一帧桌上检测）与 `pick_place(cell_id, cls)`（把指定格子的物体取走放好），自己不 `import` 任何 ROS 类型。仿真线要做的只是填好这两个缝隙的实现（图 1）：

- 缝隙 1：订阅 `/detections_d2a` (`Detection2DArray`)，把检测框中心经相机标定映射成桌面坐标与格号；
- 缝隙 2：发 `PickPlace Action`，由 `GazeboFollowExecutor` 完成六段动作并回传动作级原因。

这条“决策与通信解耦”的设计是本人定的，阶段一仿真先把它跑通；正因为缝隙两侧口径固定，后来换到真机时只换了这两个实现，契约、原因枚举、日志 schema 一行未改。

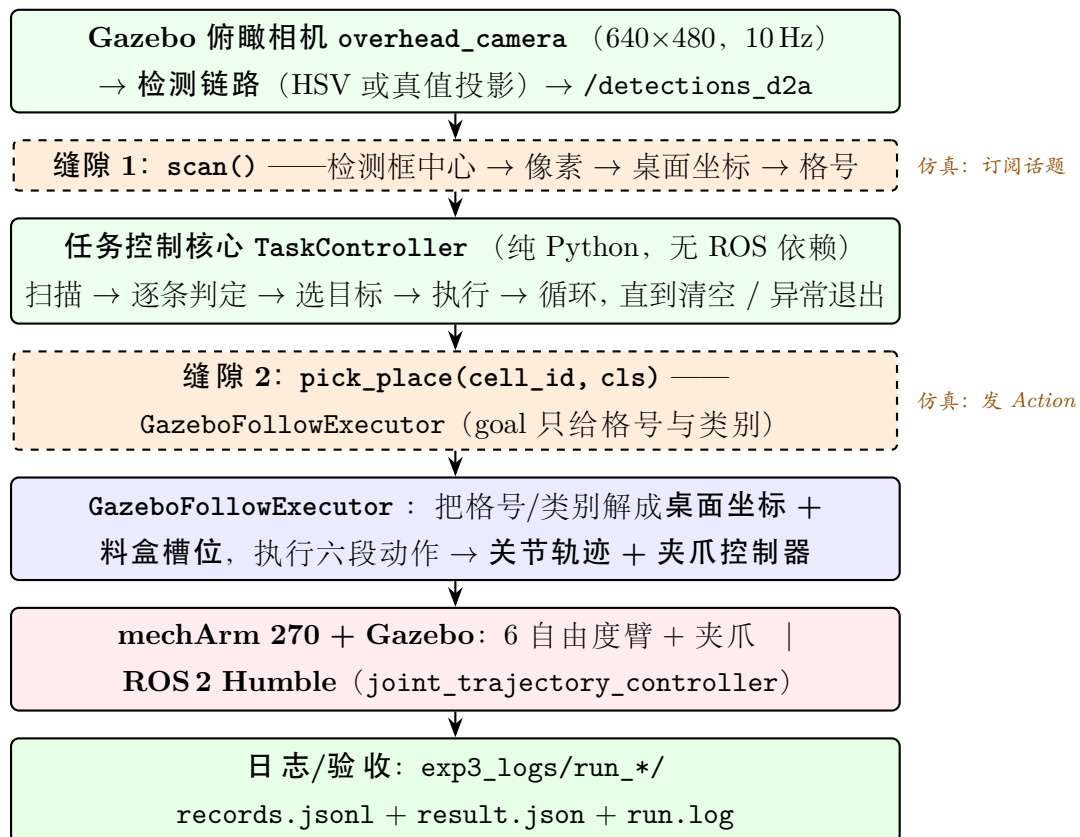


图 1: 仿真线主链与两个注入缝隙（橙色虚线）。缝两侧口径固定，故任务核心与执行侧可分别开发、失败可分别定位；这两处缝隙正是后来换到真机时唯一被替换的地方。

3.2 一次取放拆成六段：把动作序列固定，把“在哪儿”参数化

“取一个物体并放好”是一个持续数秒的长任务，服务端把它切成固定的六个阶段，每进入一段先发进度，再交给执行者 `PickExecutor`。这个抽象是相对实验二最重要的一步泛化：实验二的直驱程序是“一个固定取物点 → 固定放置点”的循环，而本实验有 6 格 × 2 料盒；若直接套用就会退化成“每格写死一个周期”。真正会变的只是“目标和料盒在哪儿”，动作序列本身完全相同——于是把序列固定成六个方法、把位置压进参数（表 1）。

段	语义	仿真侧实现	该段失败时的原因
1	下探	规划到格子 ^{上方} → 下降到抓取高度	<code>unreachable_target</code>
2	夹取	夹爪控制器闭合	<code>exec_error</code>
3	抬离	抬到搬运高度 <code>carry_z</code>	<code>grasp_failed</code>
4	搬运	横移到目标料盒槽位 ^{上方} （高度按阶梯由高到低退让）	<code>unreachable_target</code>
5	松放	下降到放置高度 → 开爪	<code>dropped_or_missed_place</code>
6	退回	抬离 → 回停靠位（ <code>DOCK_J1</code> ）	<code>unreachable_target</code>

表 1: 六段动作在仿真侧的实现，以及每段失败时映射出的动作级原因。

这张表背后有一条我给自己定的规则，值得单独说：只报这一段的代码真能做出来的判断，不替谁宣布一个没人做过的判定。两个具体后果：

- 夹取段报 `exec_error` 而不是 `grasp_failed`：这一段的代码只是合一下夹爪，从没判过“夹住没有”（判定在下一段抬离）；而 `grasp_failed` 在原因枚举里的定义恰恰是“抬离后判定没夹起”。填它等于 凭空宣布一个没做过的判定——查日志的人会以为系统真的判过。
- 搬运段与退回段报 `unreachable_target`：仿真侧有规划器，“够不着”是它给出的真实结论（逆解不收敛或撞上关节限位，如 `q5` 上限 2.0071 rad）。真机线没有规划层、报的是驱动硬错 `exec_error`——这不是没对齐，而是两个平台上“发生了什么”本来就不是同一件事。

3.3 格号怎么变成机械臂能用的坐标：相机标定与桌面坐标系

“格号 → 坐标”这一步在仿真侧用的是桌面坐标系，它由相机标定定义：

- 相机标定 (`task.json` → `camera.calibration`) 给出 `offset_px` 与 `scale_px_per_m`，把像素线性映射到 桌面坐标（米）。检测框中心先 → 像素 → 桌面坐标 → 格号，链路全程可查。

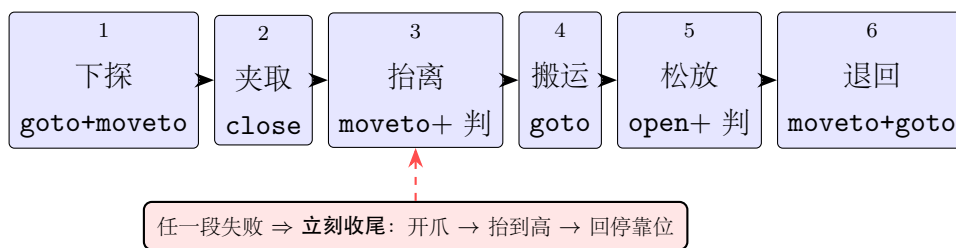


图 2: 六段动作链（仿真侧）。同一套段号也被真机线沿用，差别只在各段的原语是什么。失败段号 → 原因 → 收尾这条路径见 §3.5。

- 格号与料盒各自在 `grid_cells.json` / `bins.json` 里给出桌面坐标，执行侧只认坐标不认格号——换格子、换料盒、挪相机都只改配置，不动代码、不动接口。
- 代价（如实记录）：这套映射是线性近似（缩放 + 平移）。它在俯瞰取景下够用；若相机明显倾斜、透视不可忽略，就需要换成单应变换——本线沿用线性近似的理由是当前取景下的误差可接受，这一点有“6 格全部正确映射”的实跑结果作证 (§1)。

料盒 `bins.json` 除了中心坐标，还带落料槽位 `slots`（每个槽位一组 `dx/dy/dz` 偏移）与按料盒设定的末端偏航 `yaw_deg`——这两项是后面解决“同一料盒放多个物体”的关键 (§4 问题 3)。

3.4 检测框从哪来：HSV 与真值投影共用同一份标定（本线最关键的一处取舍）

问题的由来

仿真跑在**没有 GPU** 的云机容器里，Gazebo 走 Mesa 软件渲染 (`llvmpipe`)。用 HSV `color_detector` 跑整轮任务时，实测它吃 **479% CPU** (约 5 核)，`gzserver` 吃到 1013%；跑到第 3~4 轮就出现**画面停滞**——帧戳照常推进、画面内容却一字不变，依赖图像的检测随之失效。

做法：只换“框从哪来”，不动标定与映射

1. 新增 `truth_detector`：从 `/gazebo/model_states` 取物块世界坐标，经同一份已标定的相机参数 (§3.3) 投影为像素，再按归一化约定发出 `Detection2DArray`；
2. 下游“像素 → 桌面 → 格号”的映射一行未改，`launch` 用 `detector:=hsv|truth` 切换；
3. CPU 从 479% 降到 **4.5%**，渲染随之恢复健康——录屏轮抽 7 个时间点的帧校验，`md5` 两两全不同，**画面全程在变**。

为什么必须把话说清楚

真值投影是**理想感知**：无漏检、无噪声、无遮挡，比任何真实相机都强。因此：

- **能证明**：相机标定正确、投影与网格映射正确、规划—取放—落料—回航这条执行链可行；
- **不能声称**：HSV 检测算法在整轮任务中稳定有效——渲染停顿之后拿不到全程证据。渲染健康的第 1 轮 HSV 实测 **6/6 全部正确映射**，这份证据单独保留。

这两件事在报告里必须分开陈述。混为一谈，等于把“执行链跑通”说成了“检测算法全程有效”。

3.5 失败即收尾：段号 → 原因 → 立刻回到已知状态

一次取放是**持续数秒的长动作**。中途失败时机械臂停在一个**未定义的位置**：若只记一笔就继续下一个物体，下一次规划会**从一个非法起点出发**——轻则逆解不收敛，重则拖着臂横扫桌面。本线的处理是三步（图 3）：

1. **段号定因**：失败发生在第几段，就决定了动作级原因是哪个（表 1），日志里能直接看出“是没夹起”还是“够不着”；
2. **立刻收尾**：开爪 → 抬到高点 → 回停靠位（DOCK_J1），**尽力而为、不再抛错**（反正已经在失败路径上）；
3. **每次扫描前强制回停靠位**：臂若停在桌面中间会**挡住俯瞰相机**，实测直接造成漏检。所以收尾之后强制回停靠位，并在两次扫描之间设**最小间隔**（8s），等画面稳定下来再取帧。

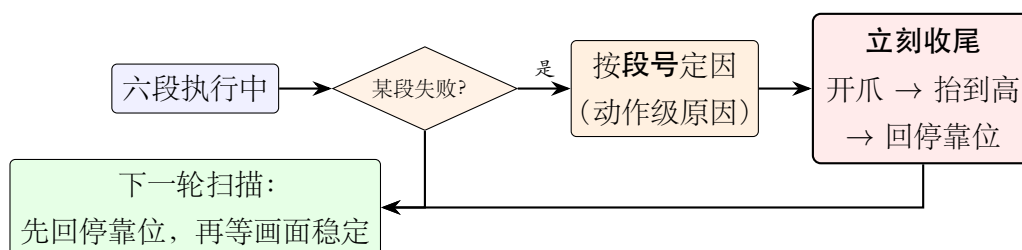


图 3: 失败路径：段号定因 → 立刻收尾 → 下一次动作前再归位。三级递进都是为了同一件事——**不让机械臂带着未知位姿继续工作**。

3.6 日志与验收口径：让“这一轮算不算”由文件说了算

仿真的日志落在 `exp3_logs/run_< 时间戳 >/: records.jsonl` 是逐物体记录（格号、类别、结果、原因），`result.json` 是整轮结论，`run.log` 是过程流水。前两份的 schema 与真机线完全相同，两线结果可以直接比对。

`result.json` 里定胜负的是这几个字段：

- `verdict`: PASS / FAIL / TRIAL——只有完整的验收轮才可能判 PASS/FAIL；场上物体不足时判 TRIAL；
- `placed_ok` / `objects_seen`: “放对了几个”与“场上共几个”；
- `reasons`: 按原因码计数（`out_of_grid`、`dropped_or_missed_place`、`no_exec` 等）。

这里有一条我特意定死的口径：空网格的判据是“没有记录”，而不是“记了一条跳过”。异常轮把 `c3` 故意空置之后，`records.jsonl` 里根本没有 `c3` 这一行，`reasons` 里也没有 `c3` 的键。这样“跳过了”这件事无法靠事后补一行来伪造——证据是那份文件的缺席，不是某一行的措辞。

仿真侧还有一点比真机干净：整轮没有任何人工参与。判类别由检测链路给、选目标与触发动作由状态机给，日志里不存在需要人回答的环节，因此也不存在“这份结果不够得上验收凭据”的争议。

4 项目中遇到的问题 and 解决方法

问题 1：搬运高度一抬高，规划就失败——而它和搬运距离无关。

定位：搬着物体横移时要抬到一个“既够高不碰桌、又够低够得着”的高度。一开始按料盒高度的比例取 `carry_z = 0.125 / 0.112m`，结果 `cup` 那 4 格全部不可达。直觉上“离得越远就抬得越高”，但逆解撞的是关节限位（`q5` 上限 `2.0071rad`），与目标料盒在 y 方向的位置有关，与搬运距离无关——这个反直觉的结论只在离线扫参时才看得出来。

解决：把可达性判断从云机搬到本地。`c4_2/core/planner.py` 的 `plan_segment` 只是 Newton + 数值 Jacobian，不碰 ROS、不碰 Gazebo，所以能离线扫。扫出的上界是：目标在 $-y$ 侧（`cup` 料盒）时 `carry_z` 上限 `0.096m`， $+y$ 侧（`mouse` 料盒）上限 `0.112m`。于是把阶梯改成由高到低退让（`0.112 → 0.096 → 0.088 → 0.073`），并且离边界留约 `8mm` 余量：瞄准边界时跟踪过冲 `1mm`，就会让下一步规划从一个非法起点出发——那样成功是偶然、失败是必然。

个人思考：“能算出来”和“能稳定跑出来”是两件事。逆解在边界上数学可解，但物理侧有跟踪误差。由此定了一条规矩：凡能被离线验证的判据，就抽到不依赖运行环境的模块里（后来摆放表、异常轮空格、高度阶梯都抽到了 `scene.py`，因而进得了离线单测）；需要连 Gazebo 才能验证的东西，一轮要几分钟，不值得用来扫参数。

问题 2：一个料盒要放 3~4 个物体，第 2 个下降时直接穿进第 1 个里。

定位：6 个物体只分 2 类，一个盒子要放 3~4 个。都瞄准料盒中心时，第 2 个物体在**下降途中**就穿进第 1 个的体内，ODE 的 LCP 求解器爆解，把已经放好的那个**炸飞**——实测 c1 被抛到 $(-75.2, 47.7)$ m，整轮就此报废。

解决：给每个料盒加**落料槽位 slots**（每个槽位一组 $dx/dy/dz$ 偏移，第二层再叠一个 dz ），并按料盒注入**末端偏航 yaw_deg**（bin_cup 取 -45° ）——料盒 $+x$ 半边在 $\theta = 0$ 时不可达，转过偏航后两个槽位全通。改完后这一项的 LCP 迭代数从 3263 掉到 0。

个人思考：“放进去”在仿真里不是几何问题而是**接触动力学**问题：物体之间不互穿不是渲染效果，是求解器能收敛的前提。而求解器一旦爆解，症状（物体乱飞、坐标离谱）与原因（落料位置重合）隔了十万八千里。**遗留：**第二层落差 $dz=0.036$ m 只比杯高 0.035 m 多 1 mm，加上放置误差仍会互穿（记分轮 LCP 冲到 122847），加大落差或让层间 xy 错开是下一步。

问题 3：Gazebo 画面停住了，但帧戳还在往前走。

定位：用 HSV 检测器跑到第 3~4 轮，扫描日志里的检测数从 6 掉到 1，且连续三轮**完全相同**；帧戳照常推进（173.9→181.4→189.1 s），画面内容却一字不变。物理侧完全正常（物块坐标与臂位姿都是实时的），所以死的是**渲染管线**，不是算法。根因是容器无 GPU：color_detector 吃 479% CPU、gzserver 吃 1013%，渲染抢不到资源。

解决与口径：见 §3.4——改用真值投影给检测框（CPU 降到 4.5%），并如实区分“证明了什么、不能证明什么”。

个人思考：只比对检测数量永远看不出来。真正一眼定案的是给扫描日志加上每个 blob 的“类别 @ 像素 → 格子”之后：同一轮里 c1 已经被取走，却仍在原像素位置被检出画面是**滞后的旧内容**。诊断日志要打**可比对的具体值**，不要只打计数。

问题 4：扫描时机械臂就横在相机视野里，白丢一半物体。

定位：俯瞰相机装在正上方，机械臂的停靠位若还在桌面中间，扫描时它就把整桌挡掉一块。表现是“明明摆着 6 个物体，只检出 4 个”，而检测算法本身没有问题——于是很容易往“阈值没调好”的方向查，越查越远。

解决：两件事必须配套做：把**停靠位移到画面边缘**（DOCK_J1 = -2.6 ），并把 motion['home'] 指向停靠位；**收尾之后强制回停靠位**，且两次扫描之间设**最小间隔**（8 s），等画面稳定下来再取帧。少做其中一件，就会分别退化“遮挡漏检 / 每轮都遮挡 / 取到上一帧的旧内容”。

个人思考：感知与执行共用同一个物理空间时，“机器人自己挡住自己的眼睛”是最容易被忽略的一类耦合。它不报错，只让成功率悄悄下降——没有“把臂挪开”这一步，前面所有调参都是在噪声里调。

问题 5：代码明明改对了，收到的却还是旧格式——因为上一轮的进程还活着。

定位：真值检测器与 HSV 检测器发的是**同一个话题** /detections_d2a。切换检测器时若只按新代码重启，上一轮那个**旧代码的进程**可能仍在发帧，两路数据混在一起。现象极具误导性：归一化约定已经改对了，日志里收到的却还是像素值 273.0。

解决：把**每一个会往同一话题发帧的进程名**都写进 `reset_scene.sh` 的清理名单，并且**连杀多轮**（`launch` 会拉一串子进程，父进程被杀后子进程仍会短暂存活，它们的定时器还会去连新 `gzserver` 并提前激活控制器）。同类地，用 `setsid` 起的记录脚本（轨迹 / 相机 / 时钟 / 录屏）**不在 `launch` 的进程树里**，也必须按名字单独清——漏掉之后它们会一轮轮堆积空转，实测三轮叠加就把 `load` 顶了上去。

个人思考：“我改的代码没生效”，有一半不是代码问题而是**有两个进程在跑**。凡存在“两个进程可能发同一路数据”的场景，清理名单就该与代码一起维护。

附：本人这条线的复现方式

```
# 1) 离线全套单测（无需 ROS、无需 Gazebo、无需相机）
cd exp3 && <项目 venv>/bin/python -m unittest discover -s tests -t .
# 260 项全绿。注意必须用项目 venv：换解释器会因缺 numpy 静默跳过 25 项

# 2) 同步权威源码 + 重建（图元化 URDF + colcon build）
cd 实验3_仿真运行证据_20260923 && bash resume.sh

# 3) 记分轮（真值检测器）
ssh autodl-exp3 'cd /root/autodl-tmp/ws_mecharm && DET=truth bash runf2.sh'

# 4) 异常轮（c3 故意空置；verdict 必为 TRIAL，与记分轮分开跑）
ssh autodl-exp3 'cd /root/autodl-tmp/ws_mecharm && \
  DET=truth LAUNCH_EXTRA="empty_cell:=c3" bash runf2.sh'

# 5) 录演示视频（订阅 /overhead/image_raw 写 mp4，不依赖 GL）
python3 cam_record.py exp3_sim_raw.mp4 --seconds 400

# 6) 清理（务必在下一轮之前跑；漏了会有旧进程继续发帧）
ssh autodl-exp3 'cd /root/autodl-tmp/ws_mecharm && bash reset_scene.sh'

# 日志：exp3_logs/run_<时间戳>/{run.log, records.jsonl, result.json}
```

说明：上列第 1 项（260 个离线用例，含摆放表、落料槽位、异常轮空格、原因映射与跳过语义）在当前仓库中可直接执行并已全绿；**第 2–6 项需要有 Gazebo 与 ROS2 的运行环境**，本报告引用的三轮结果均在其中跑出，证据落在 实验 3_ 仿真运行证据 _20260923/。相关代码在 `exp3/ros2/exp3_sim/`（场景、取放服务端、真值检测器、`launch`）与 `exp3/config/`（网格、料盒与槽位、相机标定）中。